

**Project Submission Report for Database
Management Systems (UCS310) by**

Submitted By:

Ridhi Aggarwal 1024030528

Tanishq Goyal 1024030531

Saanjal Jain 1024030533

Submitted To:

Dr. Simranjit Kaur

A Project On:

**Healthcare Inventory & Prescription
Management System**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING THAPAR
INSTITUTE OF ENGINEERING AND TECHNOLOGY,
(A DEEMED TO BE UNIVERSITY) PATIALA, PUNJAB

Jan - May 2026

Table Of Contents :

Sr. no.	Topic	Page no.
1	Project Statement	3
2	Requirement Analysis	4-5
3	Entity Relationship Diagram	6
4	ER To Table Conversion	7-9
5	Normalization	10
6	SQL Code and Procedures	11-17
7	Triggers	18-19
8	Cursors	20
9	Frontend	21-22
10	Conclusion	23
11	References	24

Problem Statement

Many government dispensaries lack a proper system to manage medicines and prescriptions, leading to serious issues such as expired medicines being dispensed, medicines being sold without valid prescriptions, and inconsistent pricing. Poor inventory tracking results in stock shortages or overstocking, while manual record-keeping increases the chances of errors and mismanagement. A centralized system is needed to ensure accurate tracking, validation, and control of healthcare data

Overview

The **Centralized Healthcare Inventory and Prescription Management System** is a database-driven solution designed using SQL and PL/SQL to efficiently manage patient records, prescriptions, and medicine inventory.

The system ensures proper prescription validation, prevents the use of expired medicines, and maintains accurate stock levels through automated processes such as triggers and constraints. By integrating all operations into a single database, the system improves efficiency, reduces errors, and enhances transparency in healthcare management.

Requirements Analysis

To develop the Centralized Healthcare Inventory and Prescription Management System, the following requirements were identified based on real-world healthcare challenges:

1. Patient, Staff & Role Management

The system must manage patient records and healthcare staff, including doctors and pharmacists. Role-based access is enforced to ensure that only authorized personnel perform specific actions.

2. Prescription & Visit Management

The system must maintain detailed records of patient visits and prescriptions. Each prescription is linked to a visit, doctor, and patient, ensuring proper traceability and authenticity.

3. Medicine & Batch-Level Inventory Management

The system must store medicines and maintain inventory at the batch level, including quantity, expiry date, and storage details. This enables accurate tracking of stock across dispensaries.

4. Expiry Control & Safety

The system must prevent the issuance of expired medicines. This is enforced using triggers that block transactions involving expired stock, ensuring patient safety.

5. Automated Stock Management

The system must automatically update inventory levels based on transactions such as stock addition, issue, return, and adjustments using triggers.

6. Transaction Tracking & Audit

The system must log all inventory-related transactions and maintain an audit trail to track stock changes, ensuring transparency and accountability.

7. Prescription Dispensing Workflow

The system must support a controlled dispensing process where medicines are issued based on prescriptions, with validation of stock availability and expiry conditions.

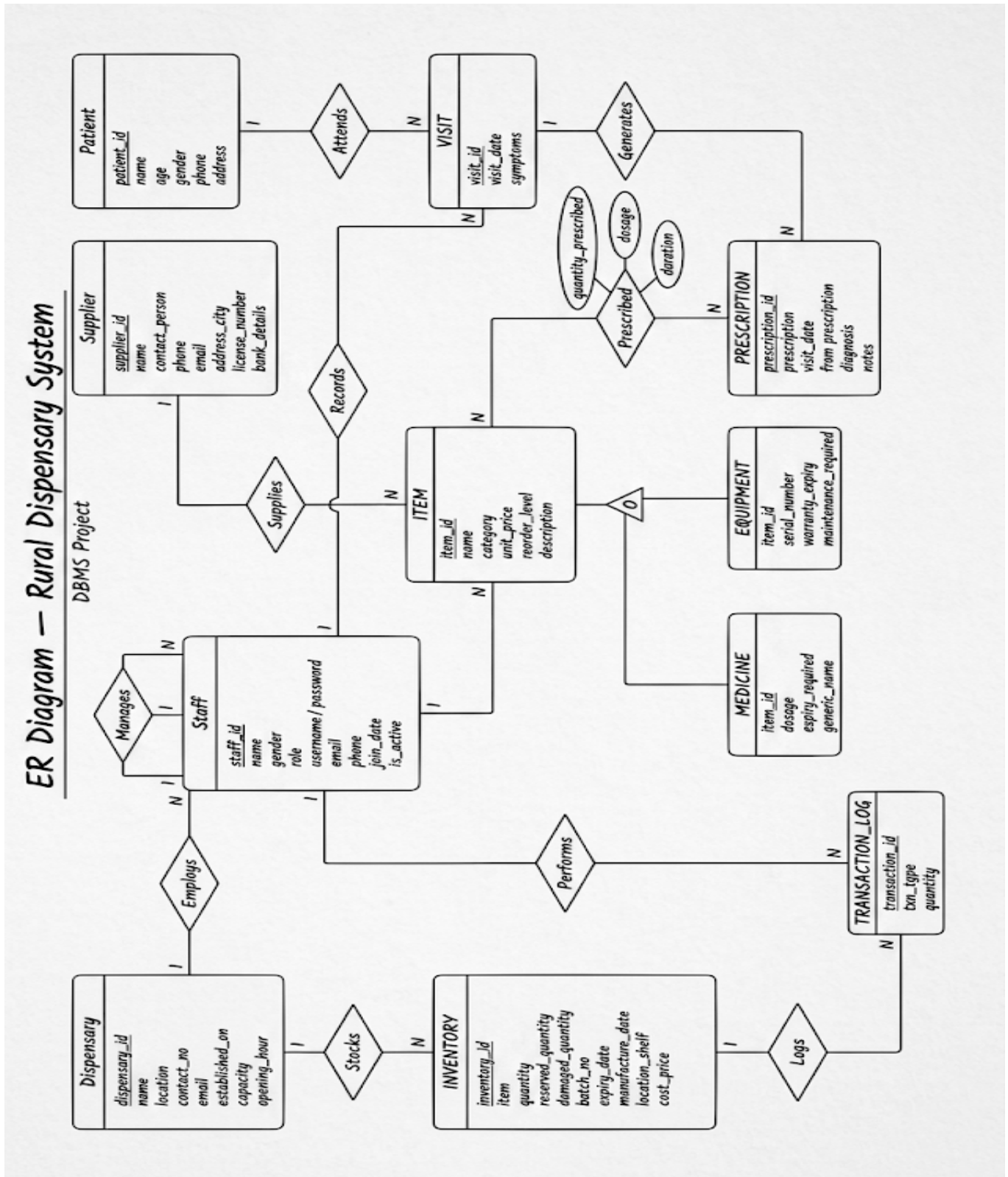
8. Data Integrity & Constraints

The system must enforce data consistency using primary keys, foreign keys, unique constraints, and validation checks.

Software Requirements

1. **SQL** – For schema design and data manipulation
2. **PL/SQL** – For triggers, procedures, and functions
3. **Oracle APEX** – Used for both backend database interaction and frontend application development
4. **Oracle Database (19c / Live SQL / XE)** – For execution and testing
5. **ER Diagram Tools (e.g., draw.io / QuickDBD)** – For system design

Entity Relationship Diagram :



ER To Table Conversion :

1. Dispensary

Column	Data Type	Key
dispensary_id	INT	PK
name	VARCHAR(100)	
location	TEXT	
contact_no	VARCHAR(15)	
email	VARCHAR(100)	
established_on	DATE	
capacity	INT	
opening_hour	VARCHAR(50)	

2. Staff

Column	Data Type	Key
staff_id	INT	PK
dispensary_id	INT	FK
name	VARCHAR(100)	
gender	CHAR(1)	
role	VARCHAR(50)	
username	VARCHAR(50)	
password	VARCHAR(255)	
email	VARCHAR(100)	
phone	VARCHAR(15)	
join_date	DATE	
is_active	BOOLEAN	

3. Inventory

Column	Data Type	Key
inventory_id	INT	PK
dispensary_id	INT	FK
item_id	INT	FK
quantity	INT	
reserved_quantity	INT	
damaged_quantity	INT	
batch_no	VARCHAR(50)	
expiry_date	DATE	
manufacture_date	DATE	
location_shelf	VARCHAR(50)	
cost_price	DECIMAL(10,2)	

4. Transaction_Log

Column	Data Type	Key
transaction_id	INT	PK
inventory_id	INT	FK
staff_id	INT	FK
txn_type	VARCHAR(50)	
quantity	INT	

5. Item

Column	Data Type	Key
item_id	INT	PK
supplier_id	INT	FK
staff_id	INT	FK
name	VARCHAR(100)	
category	VARCHAR(50)	
unit_price	DECIMAL(10,2)	
reorder_level	INT	
description	TEXT	

6. Medicine (subtype of Item)

Column	Data Type	Key
item_id	INT	PK/FK
dosage	VARCHAR(50)	
expiry_required	BOOLEAN	
generic_name	VARCHAR(100)	

7. Equipment (subtype of Item)

Column	Data Type	Key
item_id	INT	PK/FK
serial_number	VARCHAR(100)	
warranty_expiry	DATE	
maintenance_required	BOOLEAN	

8. Supplier

Column	Data Type	Key
supplier_id	INT	PK
name	VARCHAR(100)	
contact_person	VARCHAR(100)	
phone	VARCHAR(15)	
email	VARCHAR(100)	
address_city	VARCHAR(100)	

license_number	VARCHAR(50)	
bank_details	TEXT	

9. Patient

Column	Data Type	Key
patient_id	INT	PK
name	VARCHAR(100)	
age	INT	
gender	CHAR(1)	
phone	VARCHAR(15)	
address	TEXT	

10. Visit

Column	Data Type	Key
visit_id	INT	PK
patient_id	INT	FK
staff_id	INT	FK
visit_date	DATE	
symptoms	TEXT	

11. Prescription

Column	Data Type	Key
prescription_id	INT	PK
visit_id	INT	FK
prescription	TEXT	
visit_date	DATE	
from_prescription	TEXT	
diagnosis	TEXT	
notes	TEXT	

12. Prescribed (junction table)

Column	Data Type	Key
prescription_id	INT	PK/FK
item_id	INT	PK/FK
quantity_prescribed	INT	
dosage	VARCHAR(50)	
duration	VARCHAR(50)	

Normalization

Normalization is used to organize data efficiently by reducing redundancy and ensuring data integrity. The database for the Centralized Healthcare Inventory and Prescription Management System is designed up to Third Normal Form (3NF).

First Normal Form (1NF)

All tables contain atomic values with no repeating or multi-valued attributes. Each record is uniquely identified using a primary key. Tables such as Patient, Staff, Medicine, and Medicine_Inventory follow this rule.

Second Normal Form (2NF)

All non-key attributes are fully dependent on the primary key. Since most tables use single-column primary keys, partial dependency does not exist. Associative tables like Prescription_Items also maintain full dependency.

Third Normal Form (3NF)

There are no transitive dependencies. Data is properly divided into separate tables such as Patient, Visit, Prescription, Medicine, and Inventory, ensuring that non-key attributes depend only on the primary key. The Staff–Doctor–Pharmacist structure further avoids redundancy.

SQL CODE:

1.CREATING THE TABLES :

```
CREATE TABLE Medicine (  
    unit_price NUMBER(10,2) NOT NULL,  
    reorder_level NUMBER(7),  
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)  
);  
  
-- 9. Equipment  
CREATE TABLE Equipment (  
    equipment_id NUMBER(6) PRIMARY KEY,  
    supplier_id NUMBER(5),  
    name VARCHAR2(150) NOT NULL,  
    type VARCHAR2(40) NOT NULL,  
    cost NUMBER(12,2) NOT NULL,  
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)  
);  
  
-- 10. Medicine Inventory  
CREATE TABLE Medicine_Inventory (  
    inventory_id NUMBER(8) PRIMARY KEY,  
    dispensary_id NUMBER(5) NOT NULL,  
    medicine_id NUMBER(7) NOT NULL,  
    batch_no VARCHAR2(30) NOT NULL,  
    quantity NUMBER(10),  
    expiry_date DATE NOT NULL,  
    FOREIGN KEY (dispensary_id) REFERENCES Dispensary(dispensary_id),  
    FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id)  
);  
  
-- 11. Equipment Inventory  
CREATE TABLE Equipment_Inventory (  
    eq_inventory_id NUMBER(7) PRIMARY KEY,  
    dispensary_id NUMBER(5) NOT NULL,  
    equipment_id NUMBER(6) NOT NULL,  
    quantity NUMBER(5),  
    status VARCHAR2(20),  
    FOREIGN KEY (dispensary_id) REFERENCES Dispensary(dispensary_id),  
    FOREIGN KEY (equipment_id) REFERENCES Equipment(equipment_id)  
);
```

The screenshot displays a SQL IDE interface with a dark theme. At the top, a toolbar contains icons for file operations, execution, and editing. The main editor area shows a SQL script for creating a table named 'Supplier' with columns: 'supplier_id' (NUMBER(5) PRIMARY KEY), 'name' (VARCHAR2(120) NOT NULL), 'contact_person' (VARCHAR2(100)), 'phone' (VARCHAR2(15) NOT NULL), 'email' (VARCHAR2(100)), and 'city' (VARCHAR2(60)). Below the editor, a tabbed interface shows the 'Query result' tab selected, displaying the output of the SQL execution. The output shows the successful creation of the 'PRESCRIPTION' table and the 'PRESCRIPTION_ITEMS' table, along with the elapsed time for each operation. The 'Prescription' table creation took 00:00:00.015 seconds, and the 'Prescription_Items' table creation took 00:00:00.018 seconds. The 'Show more...' links are visible next to the table creation messages.

```
24 CREATE TABLE Supplier (  
25     supplier_id    NUMBER(5) PRIMARY KEY,  
26     name           VARCHAR2(120) NOT NULL,  
27     contact_person VARCHAR2(100),  
28     phone          VARCHAR2(15) NOT NULL,  
29     email          VARCHAR2(100),  
30     city           VARCHAR2(60),  
31 )
```

Query result Script output DBMS output Explain Plan SQL history

SQL> CREATE TABLE Prescription (
 prescription_id NUMBER(8) PRIMARY KEY,
 visit_id NUMBER(8) NOT NULL,
 doctor_id NUMBER(6),...
Show more...

Table PRESCRIPTION created.

Elapsed: 00:00:00.015

SQL> CREATE TABLE Prescription_Items (
 item_id NUMBER(9) PRIMARY KEY,
 prescription_id NUMBER(8),
 medicine_id NUMBER(7),...
Show more...

Table PRESCRIPTION_ITEMS created.

Elapsed: 00:00:00.018

Query result Script output DBMS output Explain Plan SQL history


```
SQL> CREATE TABLE Prescription (
    prescription_id NUMBER(8) PRIMARY KEY,
    visit_id        NUMBER(8) NOT NULL,
    doctor_id       NUMBER(6),...
```

Table PRESCRIPTION created.

Elapsed: 00:00:00.015

```
SQL> CREATE TABLE Prescription_Items (
    item_id          NUMBER(9) PRIMARY KEY,
    prescription_id  NUMBER(8),
    medicine_id      NUMBER(7),...
```

```
Table PRESCRIPTION_ITEMS created.
```

Elapsed: 00:00:00.018

2. Insertion

1 row inserted.

Elapsed: 00:00:00.002

SQL> INSERT INTO District VALUES (seq_district.NEXTVAL, 'Patiala', 'Malwa', 1895686)

1 row inserted.

Elapsed: 00:00:00.002

SQL> INSERT INTO District VALUES (seq_district.NEXTVAL, 'Jalandhar', 'Doaba', 2193590)

1 row inserted.

Elapsed: 00:00:00.002

SQL> INSERT INTO District VALUES (seq_district.NEXTVAL, 'Mohali', 'Malwa', 994628)

```
180 );
181 INSERT INTO District VALUES (1, 'Amritsar', 'Majha', 2490656);
182 INSERT INTO District VALUES (2, 'Ludhiana', 'Malwa', 3487882);
183 INSERT INTO District VALUES (3, 'Patiala', 'Malwa', 1895686);
184 INSERT INTO District VALUES (4, 'Jalandhar', 'Doaba', 2193590);
```

Query result Script output DBMS output Explain Plan SQL history

SQL> INSERT INTO District VALUES (2, 'Ludhiana', 'Malwa', 3487882)

1 row inserted.

Elapsed: 00:00:00.001

SQL> INSERT INTO District VALUES (3, 'Patiala', 'Malwa', 1895686)

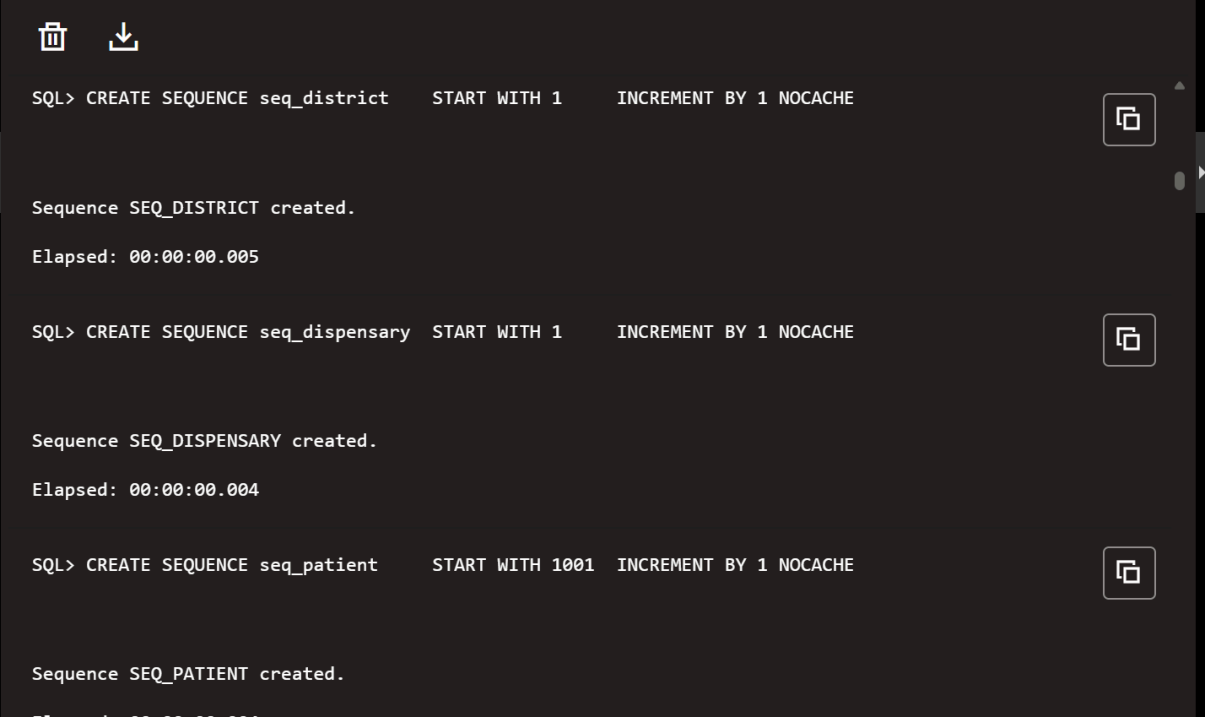
1 row inserted.

Elapsed: 00:00:00.002

SQL> INSERT INTO District VALUES (4, 'Jalandhar', 'Doaba', 2193590)

1 row inserted.

3. Sequence creation



```
SQL> CREATE SEQUENCE seq_district START WITH 1 INCREMENT BY 1 NOCACHE

Sequence SEQ_DISTRICT created.

Elapsed: 00:00:00.005

SQL> CREATE SEQUENCE seq_dispensary START WITH 1 INCREMENT BY 1 NOCACHE

Sequence SEQ_DISPENSARY created.

Elapsed: 00:00:00.004

SQL> CREATE SEQUENCE seq_patient START WITH 1001 INCREMENT BY 1 NOCACHE

Sequence SEQ_PATIENT created.
```

4. Functions

4.1 Function - 1

```
1318 ORDER BY m.name;
1319
1320 ---Q8: Function usage---Patient ages
1321 SELECT patient_id, name, fn_patient_age(patient_id) AS age
1322 FROM Patient
1323 ORDER BY age DESC;
1324
1325 Q8: View usage: Dictionary stack summary
```

	PATIENT_ID	NAME	AGE
1	1019	Satnam Toor	86
2	1037	Gurpreet Malhotra	86
3	1029	Harish Kumar	80
4	1044	Amrik Gill	79
5	1013	Dilbag Randhawa	78
6	1017	Balbir Dhillon	73
7	1042	Manmohan Bhatia	73
8	1008	Ranjit Maan	70
9	1023	Joginder Bains	69
10	1003	Amarjit Dhaliwal	66
11	1050	Rajinderpal Kundi	66

4.2 Function-2

```
1352 SET SERVEROUTPUT ON;
1353 EXEC get_patient_history(1001);
1354 SELECT fn_is_medicine_available(3001, 100) FROM dual;
```

	FN_IS_MEDICINE_A
1	0

5. Procedure

5.1 get_patient_history

Input :

```
977
978 -- PROCEDURE 1: Patient History Report
979 CREATE OR REPLACE PROCEDURE get_patient_history (p_patient_id IN NUMBER)
980 IS
981     v_patient_name VARCHAR2(100);
982     v_dob           DATE;
983
984     CURSOR cur_visits IS
985         SELECT v.visit_id, v.visit_date, v.visit_type, v.diagnosis,
986                s.name AS doctor_name, d.name AS dispensary_name
987         FROM   Visit      v
988         JOIN   Staff      s ON s.staff_id      = v.staff_id
989         JOIN   Dispensary d ON d.dispensary_id = v.dispensary_id
990         WHERE  v.patient_id = p_patient_id
991         ORDER BY v.visit_date;
992
993     CURSOR cur_presc_items (p_visit_id NUMBER) IS
994         SELECT m.name AS medicine, pi.dosage, pi.duration_days, pi.quantity
995         FROM   Prescription pr
996         JOIN   Prescription_Items pi ON pi.prescription_id = pr.prescription_id
997         JOIN   Medicine          m ON m.medicine_id      = pi.medicine_id
998         WHERE  pr.visit_id = p_visit_id;
999 BEGIN
1000     SELECT name, dob INTO v_patient_name, v_dob
1001     FROM   Patient WHERE patient_id = p_patient_id;
1002
1003     DBMS_OUTPUT.PUT_LINE('=====');
1004     DBMS_OUTPUT.PUT_LINE(' PATIENT HISTORY: ' || v_patient_name ||
1005                           ' (ID: ' || p_patient_id || ')');
1006     DBMS_OUTPUT.PUT_LINE(' DOB: ' || TO_CHAR(v_dob, 'DD-MON-YYYY') ||
1007                           ' Age: ' || FLOOR(MONTHS_BETWEEN(SYSDATE, v_dob)
1008                           / 12));
1009     DBMS_OUTPUT.PUT_LINE('=====');
1010
1011     FOR v_rec IN cur_visits LOOP
1012         DBMS_OUTPUT.PUT_LINE(
1013             CHR(10) || ' Visit #' || v_rec.visit_id || ' [' || v_rec.visit_type ||
1014             ' ' || TO_CHAR(v_rec.visit_date, 'DD-MON-YYYY') || ' at ' ||
1015             v_rec.doctor_name || ' ' || v_rec.dispensary_name || ']'
1016         );
1017         FOR cur_presc_items IN cur_presc_items(v_rec.visit_id) LOOP
1018             DBMS_OUTPUT.PUT_LINE(
1019                 CHR(10) || ' Medicine: ' || cur_presc_items.medicine ||
1020                 ' Dosage: ' || cur_presc_items.dosage ||
1021                 ' Duration: ' || cur_presc_items.duration_days ||
1022                 ' Quantity: ' || cur_presc_items.quantity
1023             );
1024         END LOOP;
1025     END LOOP;
```


Output :

```
1352 SET SERVEROUTPUT ON;
1353 EXEC get_patient_history(1001);
```

Query result Script output DBMS output Explain Plan SQL history

SQL> EXEC get_patient_history(1001)

```
=====
PATIENT HISTORY: Gurpreet Singh (ID: 1001)
DOB: 12-APR-1978   Age: 48 yrs
=====

Visit #7001   [OPD]   15-JAN-2024   at Civil Hospital Amritsar
Doctor: Dr. Harpreet Singh
Diagnosis: Viral fever
- Paracetamol 500mg | Dosage: 1 tablet TDS x 5 days | Days: 5 | Qty Issued: 15
- Cetirizine 10mg | Dosage: 1 tablet BD x 5 days | Days: 5 | Qty Issued: 10
=====
```

5.2 dispense_prescription

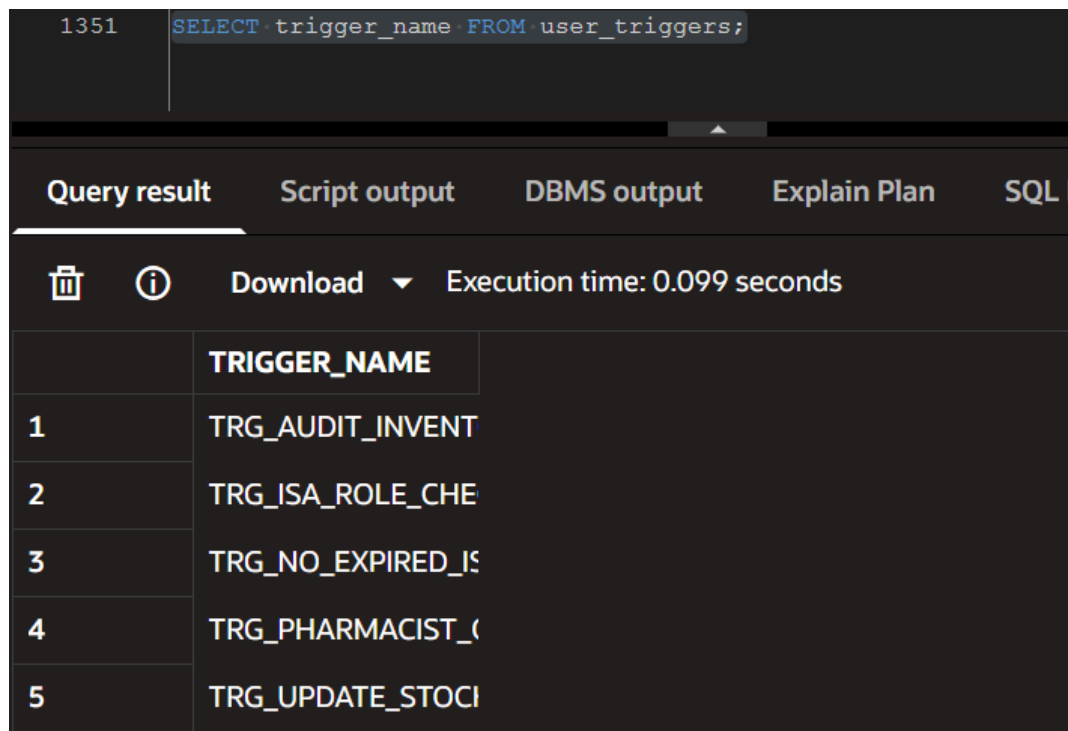
```
1354 SELECT fn_is_medicine_available(3001, 100) FROM dual;
1355 EXEC dispense_prescription(8001, 103);
1356 SELECT status FROM Prescription WHERE prescription_id = 8001;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.003 seconds

	STATUS
1	DISPENSED

6. Triggers :

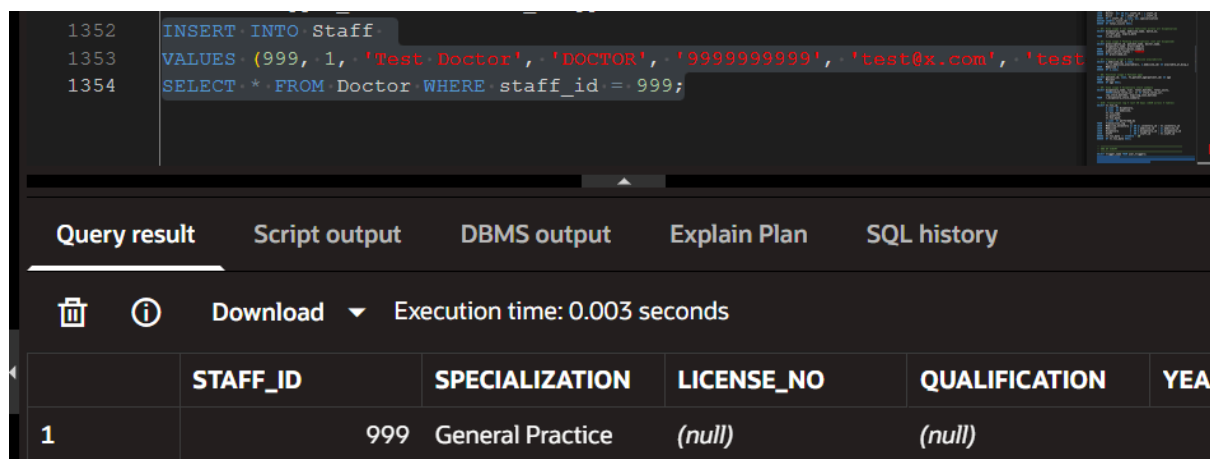


The screenshot shows a database interface with a query window at the top containing the SQL statement: `SELECT trigger_name FROM user_triggers;`. Below the query window, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is active, displaying a table with 5 rows and 1 column named 'TRIGGER_NAME'. The rows list the following trigger names: TRG_AUDIT_INVENT, TRG_ISA_ROLE_CHE, TRG_NO_EXPIRED_IS, TRG_PHARMACIST_C, and TRG_UPDATE_STOCK. The execution time is noted as 0.099 seconds.

	TRIGGER_NAME
1	TRG_AUDIT_INVENT
2	TRG_ISA_ROLE_CHE
3	TRG_NO_EXPIRED_IS
4	TRG_PHARMACIST_C
5	TRG_UPDATE_STOCK

6.1 TRG_UPDATE_STOCK

This trigger fires AFTER INSERT on Transaction_Log. When a transaction of type ISSUE is recorded, it reduces the quantity in Medicine_Inventory for the corresponding inventory_id. It ensures stock levels are always synchronized with issued medicines.

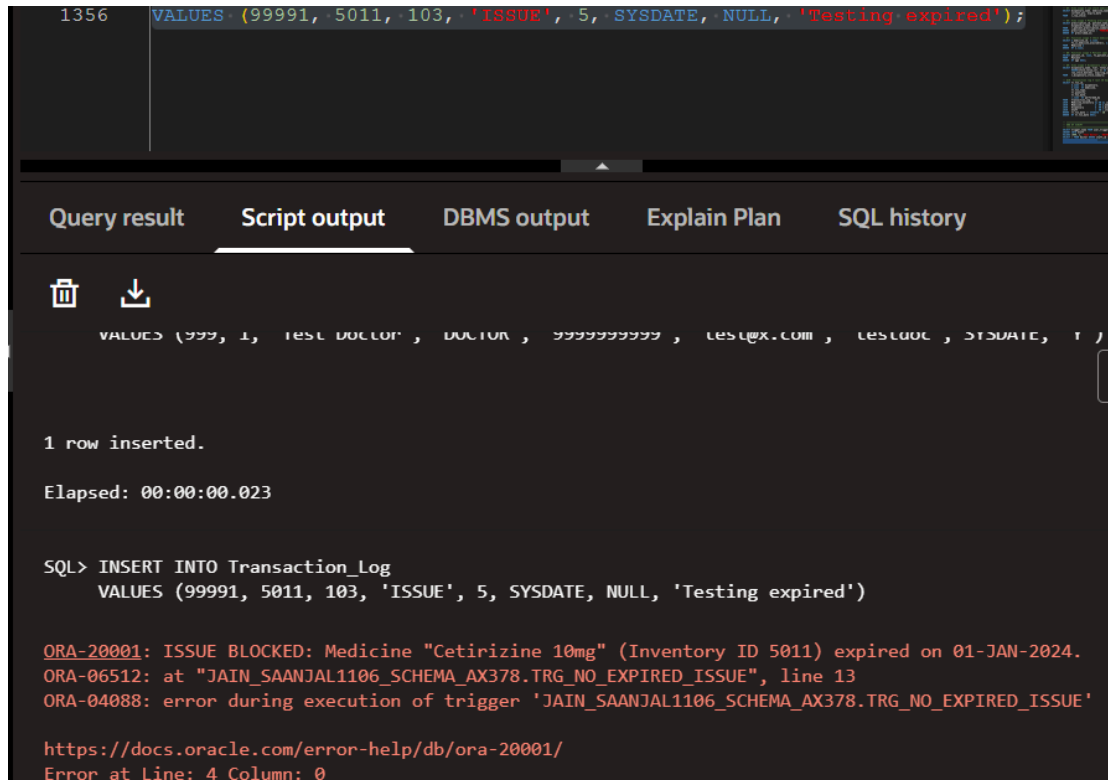


The screenshot shows a database interface with a query window at the top containing the SQL statement: `INSERT INTO Staff VALUES (999, 1, 'Test Doctor', 'DOCTOR', '999999999', 'test@x.com', 'test');`. Below the query window, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is active, displaying a table with 6 columns: STAFF_ID, SPECIALIZATION, LICENSE_NO, QUALIFICATION, and YEA. The first row shows the inserted data: STAFF_ID 999, SPECIALIZATION General Practice, LICENSE_NO (null), QUALIFICATION (null), and YEA (null). The execution time is noted as 0.003 seconds.

	STAFF_ID	SPECIALIZATION	LICENSE_NO	QUALIFICATION	YEA
1	999	General Practice	(null)	(null)	

6.2 TRG_NO_EXPIRED_ISSUE

The error occurs because a BEFORE INSERT trigger prevents issuing expired medicines by validating expiry_date and raising an exception if violated.



```
1356 VALUES (99991, 5011, 103, 'ISSUE', 5, SYSDATE, NULL, 'Testing expired');

Query result  Script output  DBMS output  Explain Plan  SQL history

VALUES (9999, 1, test DOCTOR, DOCTOR, 9999999999, test@x.com, test@oc, SYSDATE, 1)

1 row inserted.

Elapsed: 00:00:00.023

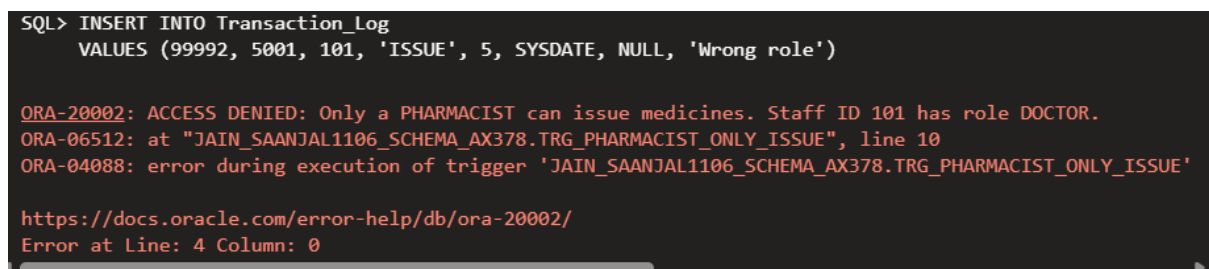
SQL> INSERT INTO Transaction_Log
VALUES (99991, 5011, 103, 'ISSUE', 5, SYSDATE, NULL, 'Testing expired')

ORA-20001: ISSUE BLOCKED: Medicine "Cetirizine 10mg" (Inventory ID 5011) expired on 01-JAN-2024.
ORA-06512: at "JAIN_SAAJAL1106_SCHEMA_AX378.TRG_NO_EXPIRED_ISSUE", line 13
ORA-04088: error during execution of trigger 'JAIN_SAAJAL1106_SCHEMA_AX378.TRG_NO_EXPIRED_ISSUE'

https://docs.oracle.com/error-help/db/ora-20001/
Error at Line: 4 Column: 0
```

6.3 TRG_PHARMACIST_ONLY_ISSUE

This trigger runs BEFORE INSERT on Transaction_Log. It verifies whether the staff_id belongs to the Pharmacist table. If not, the transaction is rejected, enforcing role-based access control.



```
SQL> INSERT INTO Transaction_Log
VALUES (99992, 5001, 101, 'ISSUE', 5, SYSDATE, NULL, 'Wrong role')

ORA-20002: ACCESS DENIED: Only a PHARMACIST can issue medicines. Staff ID 101 has role DOCTOR.
ORA-06512: at "JAIN_SAAJAL1106_SCHEMA_AX378.TRG_PHARMACIST_ONLY_ISSUE", line 10
ORA-04088: error during execution of trigger 'JAIN_SAAJAL1106_SCHEMA_AX378.TRG_PHARMACIST_ONLY_ISSUE'

https://docs.oracle.com/error-help/db/ora-20002/
Error at Line: 4 Column: 0
```

7. Cursors

7.1 Implicit Cursor

The project uses an implicit cursor within the `dispense_prescription` procedure to iterate through prescription items.

```
1407      .....WHERE pi.prescription_id = 8001
1408      .....).LOOP
1409      .....DBMS_OUTPUT.PUT_LINE (
1410      .....      'Medicine: ' || rec.medicine_id ||
1411      .....      'Quantity: ' || rec.quantity_issued
1412      .....    );
1413      .....END LOOP;
1414  END;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

```
BEGIN
  FOR rec IN (
    SELECT pi.medicine_id, pi.quantity_issued...
Show more...

Medicine: 3001 Quantity: 15
Medicine: 3010 Quantity: 10

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

Frontend:

Healthcare_Inventory_System

tgoyal1_be24@thapar.edu

Dashboard

Patients

Create Visit

Create Prescription

Prescription Items

Medicine Inventory

Add Stock

Suppliers

Medicines

Transaction Log

Patients

	Dispensary	Name	Gender	Dob	Blood Group	Phone	Address	Aadhar No	Registered On
☒	Civil Hospital Ludhiana	Kiranjit Sandhu	F	10/8/1990	AB+	9876001012	Dugri Phase 1, Ludhiana	123456780012	1/14/2024
☒	Civil Hospital Ludhiana	Ravinder Gill	M	3/25/1963	O+	9876001011	Sherpur Chowk, Ludhiana	123456780011	1/12/2024
☒	Civil Hospital Ludhiana	Diljit Hayer	M	10/17/1974	O+	9876001039	Sarabha Nagar, Ludhiana	123456780039	2/4/2024
☒	Civil Hospital Ludhiana	Sukhwinder Cheema	M	5/19/1980	B+	9876001009	Model Town, Ludhiana	123456780009	1/7/2024
☒	Civil Hospital Ludhiana	Palwinder Thind	F	9/1/1969	O+	9876001045	Jagraon, Ludhiana	123456780045	2/12/2024
☒	Civil Hospital Ludhiana	Bikramjit Hundal	M	2/7/1991	A+	9876001049	Dugri, Ludhiana	123456780049	2/20/2024
☒	Civil Hospital Ludhiana	Varinder Sran	M	5/22/1967	A-	9876001038	Focal Point, Ludhiana	123456780038	2/2/2024
☒	Civil Hospital Ludhiana	Manjit Sohal	F	12/3/1975	A+	9876001010	Haibowal Kalan, Ludhiana	123456780010	1/9/2024
☒	Civil Hospital Patiala	Tejpal Phull	M	9/9/1970	O+	9876001015	Leela Bhawan, Patiala	123456780015	1/6/2024
☒	Civil Hospital Patiala	Rajinderpal Kundi	M	6/14/1959	B-	9876001050	Rajpura, Patiala	123456780050	2/22/2024
☒	Civil Hospital Patiala	Navkiran Bedi	F	1/15/2001	A+	9876001046	Sanour, Patiala	123456780046	2/13/2024
☒	Civil Hospital Patiala	Harleen Nagra	F	11/28/1997	O+	9876001041	Bahadurgarh, Patiala	123456780041	2/6/2024
☒	Civil Hospital Patiala	Gurcharanjit Litt	M	6/3/1988	B+	9876001040	Urban Estate, Patiala	123456780040	2/5/2024
☒	Civil Hospital Patiala	Harpuneet Basra	F	12/22/1998	O-	9876001020	Nabha Rd, Patiala	123456780020	1/19/2024

Healthcare_Inventory_System

tgoyal1_be24@thapar.edu

Dashboard

Patients

Create Visit

Create Prescription

Prescription Items

Medicine Inventory

Add Stock

Suppliers

Medicines

Transaction Log

Create Prescription

Prescription

Visit

Doctor

Pharmacist

Prescribed On

Dispensed On

Status

Conclusion

This project addresses a real and important problem in healthcare systems—poor management of medicines, prescriptions, and inventory. Issues such as expired medicines, lack of proper tracking, and misuse of prescriptions can directly affect patient safety and service quality.

The Centralized Healthcare Inventory and Prescription Management System provides a structured and reliable solution to these challenges. By organizing data, automating inventory updates, and enforcing rules such as preventing expired medicine usage and restricting access to authorized staff, the system ensures safer and more efficient healthcare operations.

Using SQL, PL/SQL, and Oracle APEX, we have developed a system that reduces manual errors, improves transparency, and helps healthcare staff make better decisions. It simplifies complex processes and brings all critical information into one centralized platform.

Overall, this project is a step towards improving healthcare management by making it more accurate, secure, and dependable, ultimately contributing to better patient care

References

- Database System Concepts
Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill.
- Fundamentals of Database Systems
Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Oracle APEX
Oracle Corporation. Oracle Application Express (APEX) Documentation.
<https://apex.oracle.com>
- Oracle Live SQL
Oracle Corporation. Oracle Live SQL.
<https://livesql.oracle.com>
- Database Management Systems
Lecture notes and course materials, UCS310 – DBMS, Thapar Institute of Engineering & Technology.
- Draw.io
Used for ER diagram and database design visualization